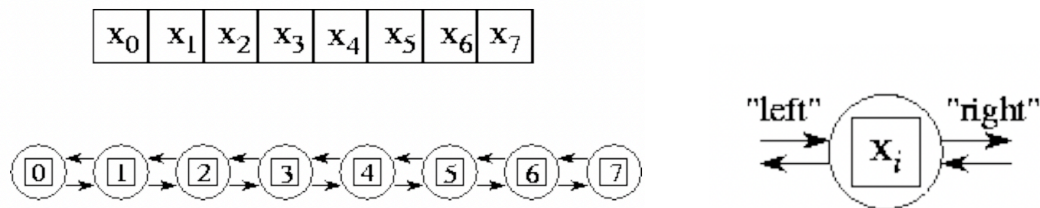




Exercice 1. Problème des différences finies en 1D

Nous considérons un problème de différences finies en une dimension, dans lequel nous avons un vecteur de taille N et devons calculer une mise à jour en utilisant une discrétisation.



1. Structure du Problème

- Nous avons un vecteur X de taille $N=8$.
- Chaque élément du vecteur correspond à une tâche indépendante, qui échange des informations avec ses voisins immédiats (gauche et droite).
- La communication se fait via des canaux, où chaque tâche reçoit et envoie des données à ses voisins.

2. Décomposition des Tâches

L'algorithme parallèle repose sur une décomposition en tâches, où chaque élément x_i du vecteur est géré par un processus indépendant t_i .

Chaque tâche t_i :

- possède deux ports d'entrée (input gauche et droite) pour recevoir les valeurs de ses voisins;
- possède deux ports de sortie (output gauche et droit) pour envoyer des valeurs mises à jour à ses voisins;
- calcule son nouveau x_i en fonction des valeurs de ses voisins x_{i-1} et x_{i+1} .

Algorithme Parallèle (Pseudo-Code)

Algorithme Différence Finie Parallèle

Entrée : Vecteur X de taille N

Sortie : Vecteur X mis à jour



-
1. Initialiser N processus $t_0, t_1, \dots, t_{N-1}$
 2. Pour chaque tâche t_i en parallèle :
 - a. Recevoir les valeurs des voisins (t_{i-1} et t_{i+1})
 - b. Calculer une mise à jour locale :
$$X_i^{(new)} = \text{fonction}(x_i) \text{ (equation (1))}$$
 - c. Envoyer $X_i^{(new)}$ à ses voisins
 3. Synchroniser les tâches
 4. Retourner le vecteur mis à jour
- Fin

Voici la formule de l'équation (1):

$$0 < i < N - 1, 0 \leq t < T : X_i^{(t+1)} = \frac{X_{i-1}^{(t)} + 2X_i^{(t)} + X_{i+1}^{(t)}}{4}.$$

Questions

1. Comment optimiser la communication entre les processus pour minimiser la latence ?
2. Pourrait-on réduire le nombre d'échanges entre les processus dans le code ?
3. Sur MPI, conseillerez-vous d'utiliser les routines de communication bloquantes pour implémenter ce problème? Pourquoi ?
4. Comment l'utilisation des routines de communications au voisinage telles que `MPI.Neighbor_alltoall()` pourrait optimiser la communication dans ce cas de figure.
5. Comparez le temps de communication et le temps de calcul. Que pouvez-vous en dire pour des valeurs de N plus grand que 8 ?
6. Proposer une implémentation de ce pseudo code sur MPI en appliquant les routines de communication discutées précédemment.

Exercice 2. Algorithme parallèle pour les interactions par paires dans un graphe

Notre deuxième exercice utilise une structure de communication similaire, mais nécessite un algorithme de communication plus complexe. De nombreux problèmes nécessitent le calcul de **toutes les interactions par paires** entre N données, soit $N \times (N-1)$ interactions.



Dans certains cas, les interactions sont **symétriques**, ce qui signifie que l'on ne doit calculer que $N(N-1)/2$ interactions distinctes. Par exemple, en **dynamique moléculaire**, on peut chercher à déterminer le **vecteur de force total** qui agit sur chaque atome, défini comme suit :

$$f_i = \sum_{j=0}^{N-1} F(X_i, X_j).$$

- Chaque **atome** est représenté par sa **masse** et ses **coordonnées cartésiennes**.
- $F(X_i, X_j)$ représente l'attraction ou la répulsion mutuelle entre deux atomes X_i et X_j .
- Dans notre exemple, $F(X_i, X_j) = -F(X_j, X_i)$, ce qui signifie que les interactions sont **symétriques**.

Algorithme parallèle simple pour les interactions par paires

Un **algorithme parallèle naïf** pour résoudre ce problème pourrait consister à créer **N tâches** :

- La tâche i reçoit une donnée locale X_i et est responsable du calcul de toutes les interactions $F(X_i, X_j)$.
- Un premier raisonnement pourrait suggérer que chaque tâche doit **communiquer avec toutes les autres**, nécessitant donc **$N \times (N-1)$ canaux** de communication.
- Cependant, une structure plus **économique** peut être utilisée avec **seulement N canaux**.

(a) Algorithme avec un anneau unidirectionnel

- Les **N tâches** sont connectées dans un **anneau unidirectionnel**, chaque tâche ayant :
 - Un **port d'entrée** (input)
 - Un **port de sortie** (output)

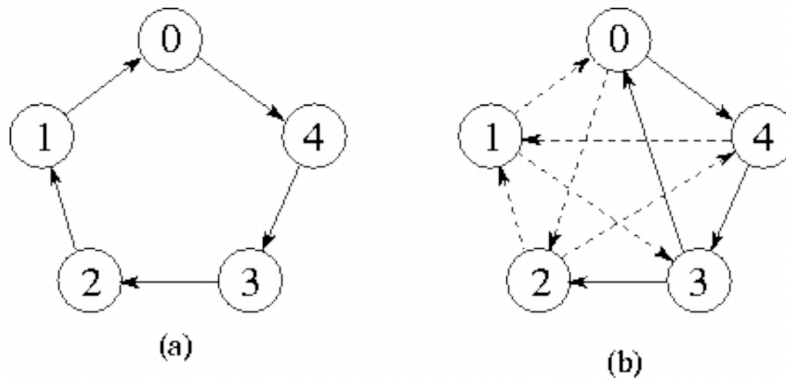


Figure 1.12: Task structures for computing pairwise interactions for $N=5$. (a) The unidirectional ring used in the simple, nonsymmetric algorithm. (b) The unidirectional ring with additional channels used to return accumulated values in the symmetric algorithm; the path taken by the accumulator used for task 0 is shown as a solid line.

Étapes de l'algorithme :

1. **Initialisation** : chaque tâche initialise un **tampon** avec la valeur de sa donnée locale et un **accumulateur** pour stocker les résultats.
2. **Répétition $N-1$ fois** :
 - **Envoi** de la valeur du tampon au voisin suivant
 - **Réception** d'une nouvelle valeur depuis le voisin précédent.
 - **Calcul** de l'interaction entre cette nouvelle valeur et la donnée locale.
 - **Mise à jour** de l'accumulateur local avec la nouvelle interaction calculée.
3. À la fin des $N-1$ itérations, chaque tâche aura **traité toutes les interactions** $F(X_i, X_j)$ impliquant sa donnée.

Questions

1. Combien de communications effectue chaque tâche ?
2. Supposons maintenant que les interactions sont symétriques, comment se comporterait la communication entre les tâches?
3. Proposer une version optimisée de l'algorithme en prenant en compte des interactions symétriques.
4. Proposer une implémentation de cette version sur MPI.



Exercice 3. Étude de Paramètres en Calcul Massivement Parallèle

Dans certains problèmes dits "**embarrassingly parallel**", chaque tâche peut s'exécuter indépendamment des autres, sans nécessiter de communication entre elles. Un exemple typique est une **étude de paramètres**, où un même calcul doit être effectué pour différentes valeurs de paramètres lues dans un fichier d'entrée, puis écrites dans un fichier de sortie après exécution.

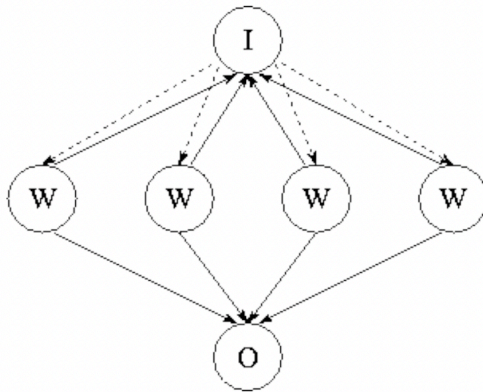


Figure 1.14: Task structure for parameter study problem. Workers (*W*) request parameters from the input task (*I*) and send results to the output task (*O*). Note the many-to-one connections: this program is nondeterministic in that the input and output tasks receive data from workers in whatever order the data are generated. Reply channels, represented as dashed lines, are used to communicate parameters from the input task to workers.

Dans cet exercice, nous allons implémenter une **distribution dynamique** des tâches de calcul sur plusieurs processeurs, en utilisant une **structure maître-esclaves** (ou **input-workers-output**).

Vous devez développer un programme parallèle qui :

1. **Lit un ensemble de paramètres d'entrée** à partir d'un fichier.
2. **Distribue dynamiquement ces paramètres** aux travailleurs (workers).
3. Chaque worker **effectue un calcul** basé sur ces paramètres.
4. **Retourne les résultats** à une tâche de sortie (output task) qui les écrit dans un fichier.

Contraintes :



-
- La distribution des tâches est dynamique : chaque worker **demande** un paramètre à traiter lorsqu'il est disponible.
 - Les résultats sont écrits dans un **ordre non déterministe**, c'est-à-dire dans l'ordre d'achèvement des calculs, et non dans l'ordre des paramètres d'entrée.
 - Pour améliorer l'efficacité, les workers peuvent **précharger** un paramètre avant d'avoir terminé leur calcul actuel (technique de **prefetching**).

Questions :

1. **Modélisation du problème :**
 - a. Décrivez comment vous pouvez organiser le travail en utilisant une approche maître-esclaves.
 - b. Expliquez comment les tâches sont attribuées dynamiquement aux travailleurs.
2. **Programmation parallèle :**
 - a. Implémentez cet algorithme en **OpenMP** ou **MPI** pour une architecture multi-cœur ou distribuée.
 - b. Quel modèle de communication (diffusion statique vs dynamique) utilisez-vous ? Pourquoi ?
3. **Optimisation et efficacité :**
 - a. Comment minimiser l'attente des travailleurs pour maximiser l'efficacité globale ?
 - b. Expliquez comment l'utilisation du **prefetching** peut améliorer les performances.
4. **Fiabilité et ordre des résultats :**
 - a. Que se passe-t-il si un worker prend plus de temps que les autres pour effectuer son calcul ?
 - b. Comment garantir que tous les résultats sont correctement stockés, même s'ils arrivent dans un ordre différent de celui des entrées ?

Bonus :



- Modifiez votre programme pour afficher le **temps total d'exécution** et la **charge de travail moyenne** par esclave.
- Expérimentez avec **différents nombres d'esclaves** et analysez l'évolution des performances.
- Que se passe-t-il si **certains calculs sont beaucoup plus longs** que d'autres ? Proposez une solution pour équilibrer la charge de travail.

Exercice 4. Méthode des Différences Finies de Jacobi en Calcul Parallèle

La **méthode de Jacobi** est une technique de différences finies utilisée pour résoudre des équations aux dérivées partielles (EDP). Elle repose sur l'actualisation répétée d'une **grille multidimensionnelle**, où chaque point est mis à jour en fonction des valeurs de ses **voisins immédiats**.

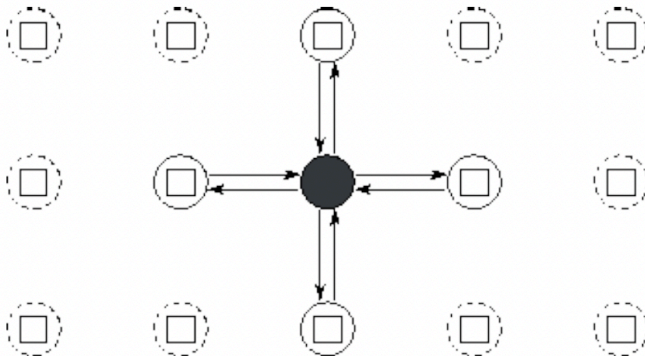


Figure 2.4: Task and channel structure for a two-dimensional finite difference computation with five-point update stencil. In this simple fine-grained formulation, each task encapsulates a single element of a two-dimensional grid and must both send its value to four neighbors and receive values from four neighbors. Only the channels used by the shaded task are shown.

Dans cet exercice, vous allez implémenter un **algorithme parallèle** pour la mise à jour d'une **grille 2D** en utilisant un **stencil à 5 points** (voisinage de Moore direct : haut, bas, gauche, droite).

Vous devez développer un programme parallèle qui :

1. Initialise une grille 2D de taille $N \times N$ avec des valeurs de départ.



2. **Met à jour chaque point** de la grille en fonction de la moyenne de ses **quatre voisins** selon la formule suivante :
$$X^{(t+1)}(i, j) = \frac{1}{4} \left(X^{(t)}(i-1, j) + X^{(t)}(i+1, j) + X^{(t)}(i, j-1) + X^{(t)}(i, j+1) \right)$$
3. **Itère cette mise à jour** un certain nombre de fois (T) ou jusqu'à convergence (erreur inférieure à un seuil donné).
4. **Optimise la communication** entre les processus pour éviter les goulots d'étranglement.

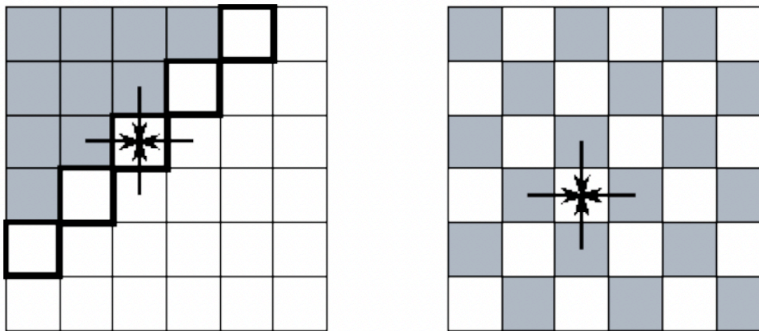


Figure 2.5: Two finite difference update strategies, here applied on a two-dimensional grid with a five-point stencil. In both figures, shaded grid points have already been updated to step $t+1$; unshaded grid points are still at step t . The arrows show data dependencies for one of the latter points. The figure on the left illustrates a simple Gauss-Seidel scheme and highlights the five grid points that can be updated at a particular point in time. In this scheme, the update proceeds in a wavefront from the top left corner to the bottom right. On the right, we show a red-black update scheme. Here, all the grid points at step t can be updated concurrently.

Questions :

1. **Modélisation du problème :**
 - a. Décrivez comment vous pouvez découper la grille pour **distribuer les calculs** entre plusieurs processeurs.
 - b. Quelle est la taille minimale de la frontière qu'un processeur doit **échanger avec ses voisins** à chaque itération ?
2. **Programmation parallèle :**
 - a. Implémentez cet algorithme en **OpenMP** (si multi-cœurs) ou **MPI** (si calcul distribué).



b. Quelle est la complexité temporelle de l'algorithme en fonction de N et T ?

3. **Optimisation et communication :**

- a. Quelle est la stratégie optimale pour **minimiser la latence** des échanges entre les processus MPI ?
- b. Que se passe-t-il si la grille est trop grande pour être stockée en mémoire locale ?

4. **Gestion des conditions aux limites :**

- a. Comment traiter les **bords de la grille** pour éviter les erreurs de mise à jour ?
- b. Que se passe-t-il si on applique des **conditions aux limites périodiques** ?

Bonus :

- **Visualisez** l'évolution de la grille à chaque itération pour observer la convergence.
- Expérimentez avec **différentes tailles de grille** et **différents nombres de processus** pour analyser l'évolutivité.
- Implémentez une version optimisée utilisant **mémoire partagée (OpenMP)** + **distribution (MPI)** en **hybride**.

Exercice 5. Réduction parallèle pour la somme de N valeurs

Implémenter un algorithme parallèle efficace pour calculer la somme de N valeurs réparties sur N tâches, en évitant les problèmes de centralisation et de séquentialité.

On considère N tâches, chacune possédant une valeur v_i . L'objectif est de calculer la somme $S = v_0 + v_1 + \dots + v_{N-1}$ de manière efficace à l'aide d'une communication en parallèle.

1. **Approche naïve :**

- Implémentez une version **centralisée** où une seule tâche (le gestionnaire) reçoit toutes les valeurs et effectue la somme.
- Mesurez le temps d'exécution et observez le goulet d'étranglement dû à la centralisation.



2.3.2 Global Communication

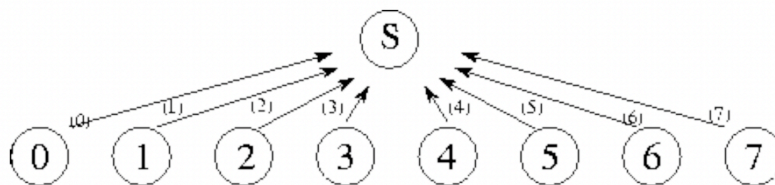


Figure 2.6: A centralized summation algorithm that uses a central manager task (S) to sum N numbers distributed among N tasks. Here, $N=8$, and each of the 8 channels is labeled with the number of the step in which they are used.

2. Approche optimisée :

- Implémentez un **algorithme distribué** en utilisant un **réseau en arbre binaire** ou une **chaîne unidimensionnelle** (Figure 2.7).
- Chaque tâche i (sauf la dernière) envoie son **résultat partiel** à son voisin de gauche.
- La tâche **0** récupère le résultat final après **$N-1$** étapes.

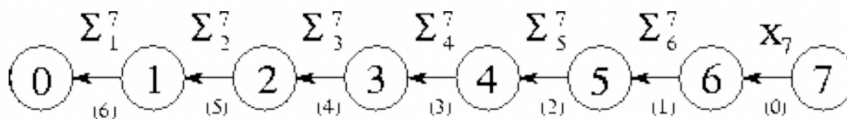


Figure 2.7: A summation algorithm that connects N tasks in an array in order to sum N numbers distributed among these tasks. Each channel is labeled with the number of the step in which it is used and the value that is communicated on it.

3. Optimisation avec une réduction binaire

- Implémentez une version où la somme est réalisée en **$\log(N)$ étapes** en divisant les tâches en paires qui s'additionnent à chaque étape (ex. **Tree-based reduction**).

Questions :

1. Pourquoi la première approche est-elle inefficace ?
2. Comparez le **nombre total de communications** et le **temps d'exécution** entre les différentes approches.
3. Comment modifier l'algorithme pour prendre en charge d'autres opérateurs associatifs (ex. multiplication, max, min) ?



-
4. Comment l'algorithme pourrait-il être adapté pour **OpenMP** ou **MPI** sur un système multi-cœur ou multi-nœuds ?

🚀 **Bonus** : Implémentez la version en utilisant **MPI_Reduce** et comparez avec votre propre version optimisée !

Exercice 6. Réduction parallèle avec Divide and Conquer

On dispose de **N** valeurs réparties sur **N** tâches. L'objectif est de calculer leur somme totale en exploitant la technique **Divide and Conquer**, qui permet d'effectuer des sommes partielles de manière concurrente.

1. **Approche séquentielle** :
 - Implémentez un algorithme simple qui additionne les **N** nombres de manière séquentielle.
 - Mesurez le temps d'exécution.
2. **Approche en arbre binaire (Binary Tree Reduction)** :
 - Réalisez une **décomposition récursive** en divisant l'ensemble des valeurs en **deux sous-ensembles de taille égale**.
 - Chaque sous-ensemble est additionné indépendamment (en parallèle).
 - Répétez cette opération **jusqu'à ce qu'il ne reste qu'une seule valeur** (la somme totale).
 - Représentez l'algorithme sous forme d'un **arbre binaire** où chaque nœud effectue une addition (Figure 2.8).
 - Implémentez l'algorithme en **parallèle** en utilisant **OpenMP** ou **MPI**.

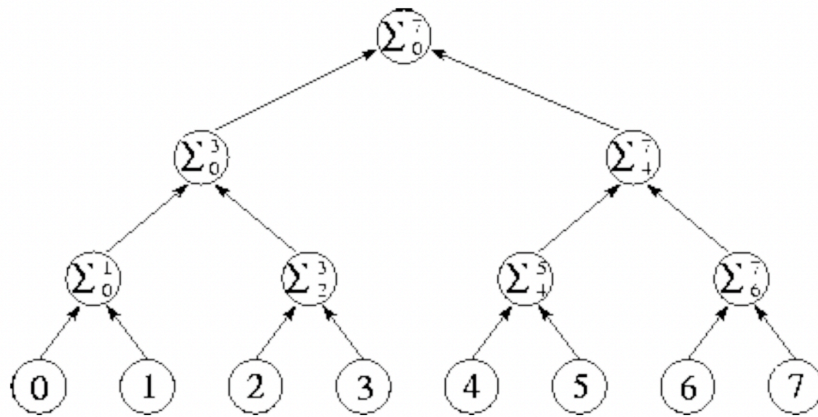


Figure 2.8: Tree structure for divide-and-conquer summation algorithm with $N=8$. The N numbers located in the tasks at the bottom of the diagram are communicated to the tasks in the row immediately above; these each perform an addition and then forward the result to the next level. The complete sum is available at the root of the tree after $\log N$ steps.

Questions :

1. Comparez l'**efficacité** de cette méthode avec celle de la somme séquentielle en mesurant le temps d'exécution pour différents N .
2. Quelle est la complexité temporelle de chaque méthode ?
3. Comment adapter cette technique pour d'autres opérations associatives comme la multiplication ou le calcul du maximum ?
4. Comment gérer le cas où N n'est pas une puissance de 2 ?



Bonus : Implémentez la version avec **MPI_Reduce** et comparez avec votre propre version optimisée !



```
procedure divide_and_conquer
begin
  if base case then
    solve problem
  else
    partition problem into subproblems  $L$  and  $R$ 
    solve problem  $L$  using divide-and-conquer
    solve problem  $R$  using divide-and-conquer
    combine solutions to problems  $L$  and  $R$ 
  endif
end
```

Algorithm 2.1 The divide-and-conquer strategy, a technique commonly used to uncover parallelism in algorithms. This procedure provides a recursive formulation of the strategy. If we are dealing with a primitive ("base case") problem, the problem is solved. Otherwise, the problem is divided into two subproblems. If these subproblems are independent, they can be solved concurrently.

Exercice 7. Communication Asynchrone et Accès Concurrent aux Données

Mettre en place un système de communication asynchrone où plusieurs tâches de calcul accèdent à une structure de données distribuée, en utilisant la seconde approche décrite (Figure 2.10).

Énoncé :

Vous devez implémenter un programme parallèle où :

- **N tâches de calcul (C)** effectuent des opérations nécessitant l'accès à une structure de données.
- **M tâches de données (D)** gèrent les accès en traitant les requêtes de lecture et d'écriture.
- La communication entre les tâches doit être **asynchrone** : une tâche de calcul envoie une requête et continue son exécution sans attendre immédiatement la réponse.

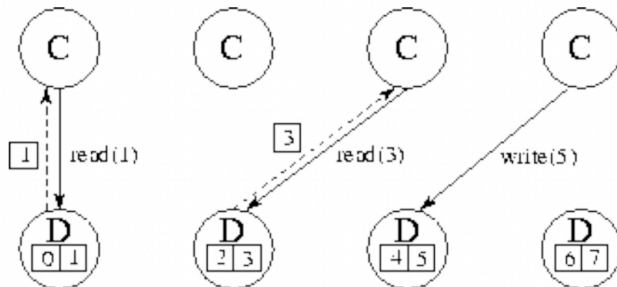


Figure 2.10: Using separate “data tasks” to service read and write requests on a distributed data structure. In this figure, four computation tasks (C) generate read and write requests to eight data items distributed among four data tasks (D). Solid lines represent requests; dashed lines represent replies. One compute task and one data task could be placed on each of four processors so as to distribute computation and data equitably.

Tâches à faire :

1. Modélisation du problème

- Représentez la structure de données sous forme de **tableau distribué** (par exemple, un dictionnaire partagé entre plusieurs tâches de données).
- Définissez un protocole d'accès (ex. : `READ(id)`, `WRITE(id, value)`) et assurez-vous qu'une tâche de calcul ne sait pas à l'avance quand elle recevra sa réponse.

2. Implémentation en Python avec MPI (Message Passing Interface)

- Utilisez `mpi4py` pour gérer la communication entre tâches de calcul et tâches de données.
- Les tâches de calcul doivent envoyer des requêtes aux tâches de données et poursuivre leur exécution en attendant la réponse via un mécanisme asynchrone (`MPI.Isend`, `MPI.Irecv`).

3. Expérimentation et Analyse

- Exécutez le programme avec différents nombres de tâches de calcul et de données.
- Comparez le temps d'exécution entre une approche **synchronisée** et une approche **asynchrone**.
- Analysez l'impact de la latence des communications sur les performances.



 **Bonus :** Implémentez une version optimisée où chaque tâche de calcul prédit ses besoins en données et précharge certaines valeurs pour réduire les temps d'attente !

Exercice 8. Agglomération et Optimisation de la Concurrency

Objectif :

Comprendre et appliquer la phase d'agglomération dans la conception d'algorithmes parallèles afin d'améliorer l'efficacité de l'exécution sur une architecture parallèle spécifique.

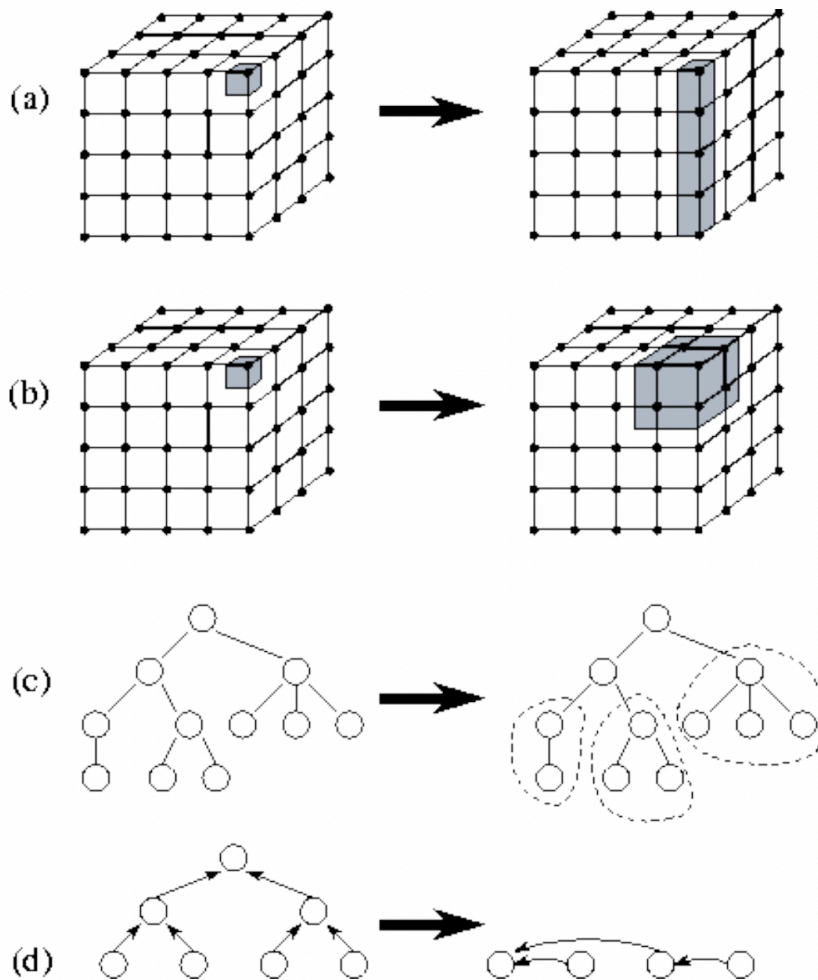


Figure 2.11: Examples of agglomeration. In (a), the size of tasks is increased by reducing the dimension of the decomposition from three to two. In (b), adjacent tasks are combined to yield a three-dimensional decomposition of higher granularity. In (c), subtrees in a divide-and-conquer structure are coalesced. In (d), nodes in a tree algorithm are combined.

Énoncé :

Vous devez optimiser un algorithme parallèle en appliquant la technique d'**agglomération**. L'algorithme initial suit une approche naïve où chaque unité de calcul est traitée indépendamment, générant ainsi un nombre excessif de tâches.

Votre objectif est de **regrouper** les tâches de manière stratégique afin de :



1. **Réduire la surcharge de communication** en augmentant la granularité des tâches.
2. **Maximiser l'efficacité sur un nombre donné de processeurs.**
3. **Assurer une bonne évolutivité** en permettant une répartition flexible des tâches.

Cas d'étude : Calcul de la somme d'un grand tableau en parallèle

1. **Approche initiale (partitionnement de base)**
 - Le tableau est divisé en N parties, chaque élément étant traité séparément.
 - Chaque tâche effectue une somme locale et envoie le résultat à une tâche maître qui effectue la somme finale.
2. **Problème identifié :**
 - Communication excessive entre les tâches et le maître.
 - Séquentialisation de la somme finale.
 - Mauvaise répartition du travail si le nombre de tâches dépasse celui des processeurs disponibles.
3. **Optimisation par agglomération :**

Groupez plusieurs éléments du tableau dans des sous-blocs plus grands.

 - Attribuez chaque sous-bloc à un processeur, qui calcule une somme partielle.
 - Effectuez une **réduction en arbre** (tree reduction) pour réduire le nombre d'étapes.
 - Comparez une approche avec et sans réplication des données.

Instructions :

1. Implémentez une version naïve de la somme parallèle.
2. Optimisez cette version en appliquant l'agglomération et en limitant les communications inutiles.
3. Expérimentez avec différents nombres de tâches et de processeurs.
4. Analysez l'impact sur le temps d'exécution et la communication.

Questions :

1. Quelle est la réduction du nombre de communications après agglomération ?



-
2. Comment choisir la taille optimale d'une tâche après regroupement ?
 3. Comment l'agglomération affecte-t-elle l'évolutivité de l'algorithme ?
 4. Peut-on appliquer cette technique à d'autres algorithmes de calcul parallèle ?

 **Bonus :** Implémentez l'algorithme en utilisant **MPI (mpi4py)** et comparez les performances avant et après l'agglomération !

Exercice 9. Optimisation par Augmentation de la Granularité

Lors de la phase de partitionnement, on cherche à découper un problème en un grand nombre de petites tâches parallélisables. Cependant, une trop grande fragmentation peut nuire aux performances à cause des coûts de communication. Cet exercice vous amène à analyser ce phénomène et à proposer une solution optimisée.

Dans un problème de différences finies en 2D (voir Figure 2.12), chaque point de la grille effectue un calcul nécessitant des données de ses voisins. Comparez deux stratégies :

- (a) Chaque point est une tâche.
- (b) Un groupe de 16 points forme une tâche unique.
- Calculez le nombre total de communications et le volume de données échangé.
- Quel scénario minimise le coût de communication tout en exploitant la parallélisation ?

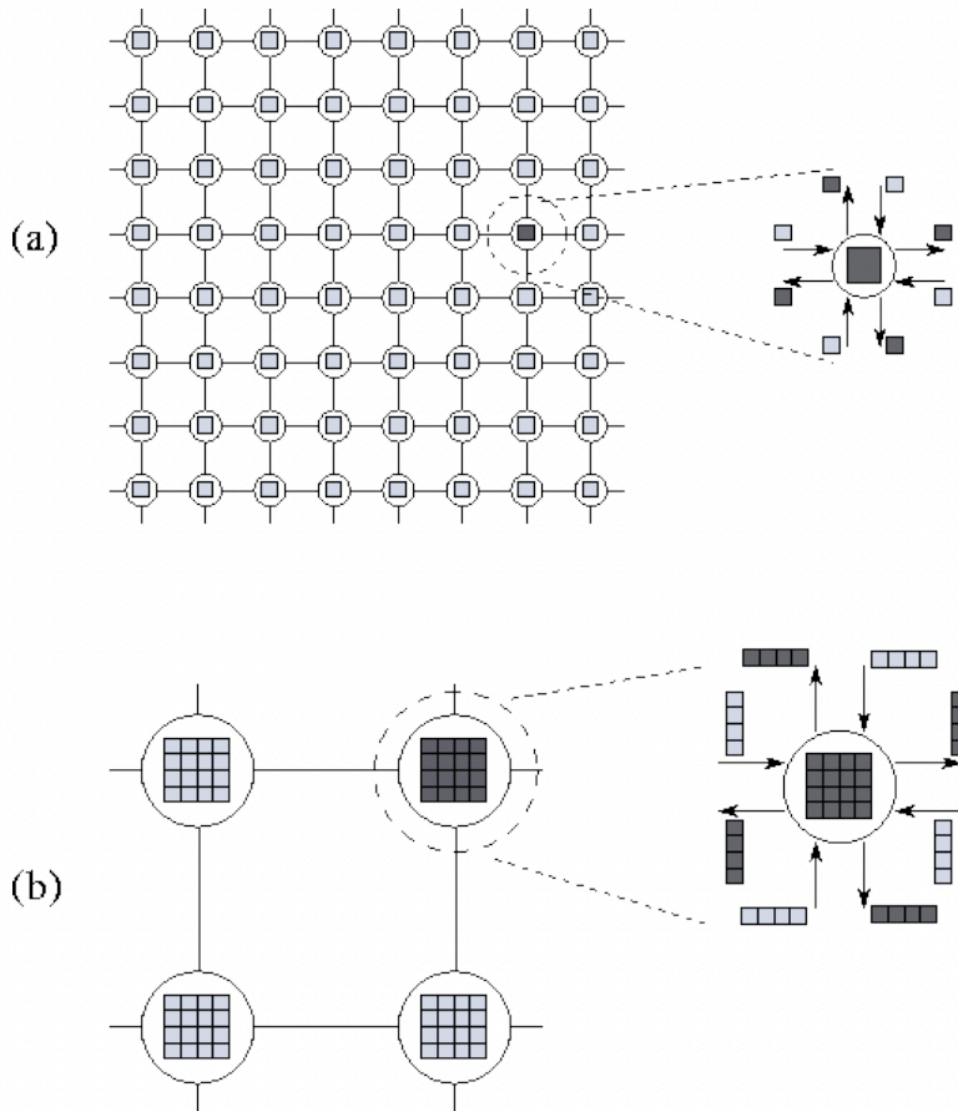


Figure 2.12: Effect of increased granularity on communication costs in a two-dimensional finite difference problem with a five-point stencil. The figure shows fine- and coarse-grained two-dimensional partitions of this problem. In each case, a single task is exploded to show its outgoing messages (dark shading) and incoming messages (light shading). In (a), a computation on an 8×8 grid is partitioned into $8 \times 8 = 64$ tasks, each responsible for a single point, while in (b) the same computation is partitioned into $2 \times 2 = 4$ tasks, each responsible for 16 points. In (a), $64 \times 4 = 256$ communications are required, 4 per task; these transfer a total of 256 data values. In (b), only $4 \times 4 = 16$ communications are required, and only $16 \times 4 = 64$ data values are transferred.

Tâches à faire



-
- Proposer une approche pour regrouper les tâches de manière optimale.
 - Comment ajuster ce regroupement en fonction du nombre de processeurs disponibles ?
 - Implémentez un algorithme en Python ou MPI qui met en œuvre cette stratégie.
-

Exercice 10. Réplication du Calcul pour Réduire la Communication

Énoncé :

Dans certains cas, il est plus efficace de dupliquer un calcul plutôt que de transmettre des données. Cet exercice explore cette idée en comparant différentes stratégies pour propager une somme totale à plusieurs tâches.

Objectifs :

- Étudier l'impact de la réplication sur la communication et le temps d'exécution.
- Comparer les stratégies classiques (diffusion) et les stratégies optimisées (réplication).
- Implémenter et tester ces stratégies sur une machine parallèle.

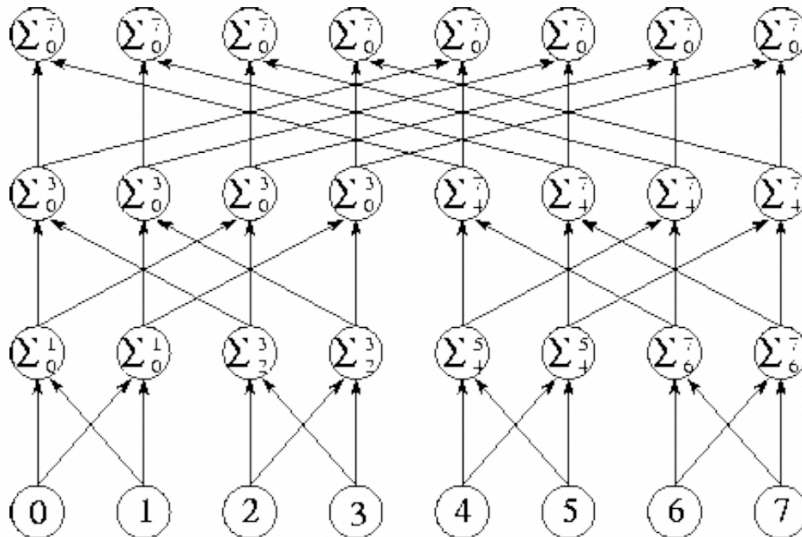


Figure 2.14: The butterfly communication structure can be used to sum N values in $\log N$ steps. Numbers located in the bottom row of tasks are propagated up through $\log N$ intermediate stages, thereby producing the complete sum in each task in the top row.

Tâches à faire :

1. Comparaison des stratégies :

- Stratégie 1 : Une seule tâche effectue la somme et diffuse le résultat via un broadcast.
- Stratégie 2 : Utilisation d'un **anneau** où chaque tâche passe le résultat à la suivante.
- Stratégie 3 : Algorithme **papillon** (Figure 2.14), où la somme se propage de manière exponentielle.
- Analysez la complexité en nombre d'étapes et en volume de communication pour chaque stratégie.
- Implémentez ces stratégies avec MPI et mesurez leur performance pour $N = 8, 16, 32, 64$ tâches.
- Quelle approche est la plus rapide et pourquoi ?

2. Application à un cas réel :

- Imaginez un scénario où la réplification de calcul serait utile (ex. simulation numérique, machine learning distribué).
- Expliquez comment cette technique pourrait améliorer l'efficacité.



Exercice 11. Optimisation des Structures de Communication : Papillon vs. Arbre

Les algorithmes en **arbre** et en **papillon** sont souvent utilisés pour agréger et diffuser des données efficacement dans les systèmes parallèles. Cet exercice vise à comparer ces deux approches et à implémenter un algorithme optimisé.

Objectifs :

- Comprendre la différence entre les algorithmes **arbre**, **anneau** et **papillon**.
- Étudier la complexité de communication et les coûts associés.
- Implémenter et comparer ces structures en utilisant un langage parallèle (MPI, OpenMP).

Questions :

1. Théorie :

- Expliquez comment fonctionne chaque structure (arbre, anneau, papillon).
- Comparez leur complexité en termes de nombre de communications et de temps d'exécution.

2. Implémentation :

- Écrivez un programme en MPI qui :
 - Effectue une somme parallèle sur **N** tâches en utilisant un **arbre**.
 - Implémente la même opération avec une **structure papillon**.
- Mesurez le temps d'exécution et comparez les résultats.

3. Optimisation et généralisation :

- Pour un nombre arbitraire de tâches, comment choisir la meilleure structure ?
- Appliquez cette logique à un problème pratique comme **l'agrégation de logs** dans un système distribué.

Exercice 12. Implémentation d'une Sommation Arborescente avec MPI

Objectif :



Implémenter une **réduction par sommation** en parallèle en utilisant **une structure d'arbre binaire** avec **MPI**.

Consigne :

1. **Initialisation des données** : Chaque processus possède un unique nombre aléatoire.
2. **Sommation arborescente** : Les processus échangent leurs données de manière hiérarchique, en suivant un schéma d'arbre binaire, jusqu'à ce que le processus **0** obtienne la somme totale.
3. **Optimisation** : Proposez une variante où certains processus effectuent des calculs localement avant de communiquer (agglomération).

1. Code de Base : Sommation Arborescente avec MPI

Ce code réalise une **réduction manuelle** en suivant une structure **d'arbre binaire**.



python

Copier

Modifier

```
from mpi4py import MPI
import numpy as np

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

# Chaque processus génère un nombre aléatoire
np.random.seed(rank) # Pour assurer la reproductibilité
value = np.random.randint(1, 10)
print(f"Process {rank} a la valeur {value}")

step = 1
while step < size:
    partner = rank + step
    if rank % (2 * step) == 0:
        if partner < size:
            received_value = comm.recv(source=partner)
            value += received_value
        else:
            target = rank - step
            comm.send(value, dest=target)
            break
    step *= 2

# Le processus 0 affiche la somme finale
if rank == 0:
    print(f"Somme totale : {value}")
```

2. Variante : Agglomération des Tâches

Au lieu de communiquer à chaque étape, nous pouvons **agglomérer les calculs** avant la communication. Voici une version optimisée :



python

Copier

Modifier

```
from mpi4py import MPI
import numpy as np

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

# Agglomération : chaque processus génère plusieurs valeurs au lieu d'une seule
local_values = np.random.randint(1, 10, size=2) # 2 valeurs par processus
local_sum = sum(local_values)

print(f"Process {rank} a les valeurs {local_values} -> somme locale {local_sum}")

step = 1
while step < size:
    partner = rank + step
    if rank % (2 * step) == 0:
        if partner < size:
            received_value = comm.recv(source=partner)
            local_sum += received_value
        else:
            target = rank - step
            comm.send(local_sum, dest=target)
            break
    step *= 2

if rank == 0:
    print(f"Somme totale optimisée : {local_sum}")
```

3. Comparaison avec **MPI_Reduce**

MPI fournit une fonction **MPI_Reduce** pour éviter d'implémenter manuellement la réduction. Essayez d'exécuter cette version et comparez avec les implémentations précédentes :



python

Copier

Modifier

```
global_sum = comm.reduce(local_sum, op=MPI.SUM, root=0)

if rank == 0:
    print(f"Somme totale avec MPI_Reduce : {global_sum}")
```

Questions :

1. Quelle est la complexité en temps et en communications de chaque approche ?
2. Pourquoi l'agglomération réduit-elle le nombre de communications ?
3. Dans quels cas l'utilisation de **MPI_Reduce** est-elle préférable à une implémentation manuelle ?

Exercice 13. Stratégies de Mapping pour un Problème de Simulation en Grille

On considère une grille $N \times N \times N$ dans laquelle chaque cellule représente une unité de calcul devant être exécutée sur un système parallèle à PPP processeurs. Chaque cellule effectue un calcul indépendant, mais doit échanger des informations avec ses 4 voisins immédiats (haut, bas, gauche, droite) à chaque itération.

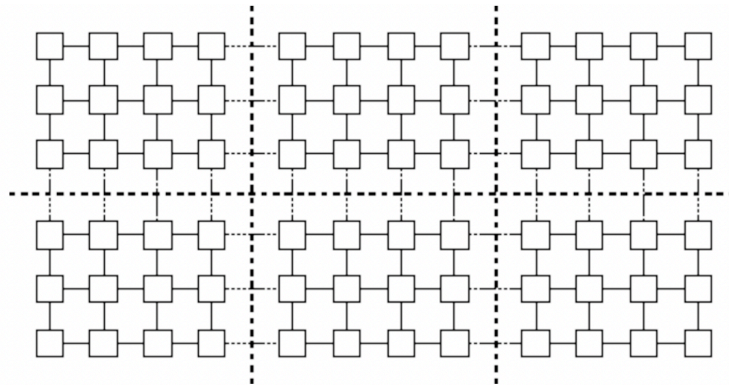


Figure 2.16: Mapping in a grid problem in which each task performs the same amount of computation and communicates only with its four neighbors. The heavy dashed lines delineate processor boundaries. The grid and associated computation is partitioned to give each processor the same amount of computation and to minimize off-processor communication.

1. Partitionnement de la grille

- Proposez une méthode de partitionnement de la grille entre les PPP processeurs pour minimiser la communication entre processeurs.
- Comparez une approche en bandes horizontales, en bandes verticales et en blocs carrés. Quelle approche minimise la communication inter-processeurs ?

2. Impact du Load Balancing

Supposons que certaines cellules nécessitent plus de calculs que d'autres, entraînant un déséquilibre de charge.

- Expliquez pourquoi un partitionnement initial basé uniquement sur la répartition égale des cellules peut poser problème.
- Proposez une stratégie de rééquilibrage de la charge (load balancing) qui minimise le coût de réallocation des tâches entre processeurs.

3. Mapping Dynamique

Dans un scénario où la charge de calcul d'une cellule varie au cours du temps,

- Quelle technique de mapping recommanderiez-vous : statique ou dynamique ? Justifiez votre choix.
- Décrivez une méthode pour détecter un déséquilibre de charge et redistribuer dynamiquement les tâches.



4. Approche Heuristique

Étant donné que le problème de mapping est NP-complet, proposez une approche heuristique permettant de trouver une solution efficace en un temps raisonnable.

Bonus : Implémentez une simulation simple en Python où une grille est répartie entre plusieurs "processeurs" (threads ou processus) et où un mécanisme de rééquilibrage est appliqué en cas de surcharge.

Exercice 14. Partitionnement et Load Balancing pour un Problème de Simulation en Grille

Vous devez répartir une grille $N \times N$ entre P processeurs pour une simulation numérique. Chaque cellule représente une unité de calcul qui échange des informations avec ses voisins immédiats. L'objectif est de minimiser le déséquilibre de charge et la communication inter-processeurs.

Questions :

1. Partitionnement Initial :

- Décrivez comment appliquer l'algorithme de *recursive coordinate bisection (RCB)* pour diviser la grille en PPP sous-domaines.
- Expliquez pourquoi cette méthode peut générer des sous-domaines « longs et fins » et l'impact sur la communication.

2. Optimisation du Partitionnement :

- Comment la méthode *unbalanced recursive bisection (URB)* améliore-t-elle la répartition des sous-domaines ?
- Comparez RCB et URB en termes de coût computationnel et d'optimisation de la communication.

3. Partitionnement pour Grilles Non Structurées :

- Dans le cas d'une grille non régulière, expliquez pourquoi la *recursive graph bisection (RGB)* est préférable à RCB.
- Décrivez le processus permettant de trouver les deux extrémités d'un graphe et d'assigner les nœuds aux sous-domaines correspondants.

4. Comparaison et Choix d'Algorithme :

- Si vous deviez équilibrer la charge sur une grille évolutive, quel algorithme



choisiriez-vous ?

b) Justifiez votre choix en considérant la variabilité de la charge et la nécessité d'un rééquilibrage dynamique.

plaintext

 Copier

 Modifier

```
Fonction RCB(Grille, P)
  Si P = 1 alors
    Assigner toute la grille à un processeur
    Retourner
  Fin Si

  Sélectionner la dimension la plus longue (X ou Y)
  Trouver la médiane de cette dimension
  Diviser la grille en deux sous-domaines

  RCB(Sous-domaine_1, P/2)
  RCB(Sous-domaine_2, P/2)
Fin Fonction
```

Algorithme 1 : Bisection Coordonnée Récursive (RCB)



plaintext

 Copier

 Modifier

```
Fonction URB(Grille, P)
  Si P = 1 alors
    Assigner toute la grille à un processeur
    Retourner
  Fin Si

  Sélectionner la dimension la plus longue (X ou Y)

  Pour chaque ratio possible (1/P, 2/P, ..., (P-1)/P) faire
    Calculer le rapport d'aspect des sous-domaines
  Fin Pour

  Sélectionner le ratio minimisant le rapport d'aspect
  Découper la grille selon ce ratio

  URB(Sous-domaine_1, P1) // P1 et P2 sont choisis dynamiquement
  URB(Sous-domaine_2, P2)
Fin Fonction
```

Algorithme 2 : Bisection Récursive Non-Équilibrée (URB)



plaintext

Copier

Modifier

```
Fonction RGB(Graphe, P)
  Si P = 1 alors
    Assigner tout le graphe à un processeur
    Retourner
  Fin Si

  Trouver les deux nœuds les plus éloignés dans le graphe
  Assigner chaque nœud au sous-domaine le plus proche de ces deux nœuds

  RGB(Sous-domaine_1, P/2)
  RGB(Sous-domaine_2, P/2)
Fin Fonction
```

Algorithme 3 : Bisection Graphique Récursive (RGB)

Bonus (Programmation) :

Implémentez un algorithme simple de **recursive coordinate bisection** en Python pour diviser une grille $N \times N$ en P sous-domaines. Ensuite, modifiez votre code pour intégrer une **unbalanced recursive bisection** et comparez les résultats.

Exercice 15. Répartition de Charge dans une Simulation sur Grille

On considère une simulation numérique sur une grille $N \times N$ représentant, par exemple, une simulation météorologique. Chaque point de la grille représente une unité de calcul et peut être considéré comme une tâche. La charge de calcul d'un point peut varier dans le temps (par exemple, à cause de l'apparition d'un « hot spot »). La grille est distribuée sur P processeurs.

Trois stratégies de répartition de charge sont envisagées :

1. **Algorithme local de load balancing** : Les processeurs comparent périodiquement leur charge avec celle de leurs voisins et transfèrent des points de grille si un déséquilibre est détecté.



2. **Méthode probabiliste** : Les tâches sont réparties aléatoirement sur les processeurs.
3. **Mapping cyclique** : Les tâches sont attribuées de façon cyclique, c'est-à-dire qu'en parcourant la liste des tâches, on assigne chaque tâche au processeur d'indice (Index mod P).

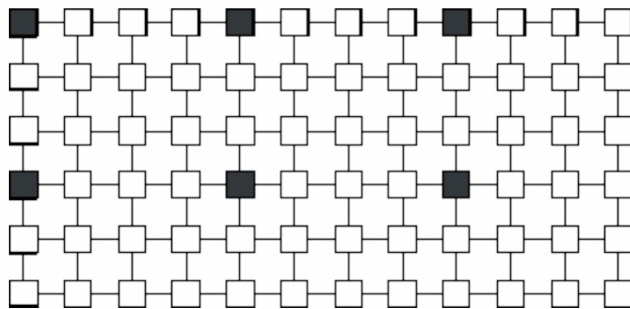


Figure 2.18: Using a cyclic mapping for load balancing in a grid problem, when executing on 12 processors. Tasks mapped to a single processor are shaded. Notice that with this mapping, all communications are with tasks located on different processors (assuming a five-point stencil).

Questions

1. Rédigez un pseudo-code décrivant le fonctionnement de l'algorithme local. Votre algorithme devra, pour chaque processeur, comparer la charge locale avec celle de ses voisins dans un maillage logique et transférer des points de grille si la différence dépasse un seuil donné.
2. Discutez des avantages et des inconvénients de cette approche locale dans le cas où la charge varie rapidement (par exemple, en présence d'un « hot spot » soudain).
3. Écrivez un pseudo-code simple qui attribue aléatoirement chaque point de la grille à un processeur parmi les P disponibles.
4. Expliquez dans quelles situations cette méthode est efficace, en prenant en compte le nombre total de tâches par rapport au nombre de processeurs, et les implications en termes de communication entre processeurs.



-
5. Rédigez un pseudo-code pour le mapping cyclique. Chaque tâche, selon son indice dans une énumération préalablement définie (par exemple, en parcourant la grille ligne par ligne), sera affectée au processeur i tel que $i = (\text{Index de la tâche}) \bmod P_i$.
 6. Analysez les avantages de cette approche en termes d'équilibre de charge, mais également les inconvénients liés à la perte de localité et l'augmentation potentielle de la communication inter-processeurs.
 7. Comparez qualitativement les trois méthodes en termes de rapidité de convergence vers un équilibre de charge, coût de communication et scalabilité.
 8. Dans quel scénario (charge homogène vs charge très variable) recommanderiez-vous chacune de ces approches ? Justifiez vos réponses.

Partie Bonus : Simulation en Python

Implémentez une simulation simple en Python qui réalise les trois stratégies de répartition sur une grille simulée. Votre simulation devra :

- Générer une grille avec une répartition initiale de charges (par exemple, avec un « hot spot » localisé).
- Appliquer l'algorithme de répartition locale, puis mesurer et afficher la charge sur chaque processeur.
- Appliquer la méthode probabiliste et la répartition cyclique, et comparer les distributions de charge obtenues ainsi que le nombre de communications nécessaires (par exemple, en comptant les transferts ou échanges inter-processeurs).

Exercice 16. Planification des Tâches dans un Système Distribué

On considère un système distribué dans lequel des tâches indépendantes doivent être exécutées par plusieurs processeurs. Chaque tâche représente un problème à résoudre et doit être attribuée à un processeur disponible. Pour cela, on utilise différentes stratégies de planification des tâches :

1. **Manager/Worker** : Un gestionnaire central attribue les tâches aux processeurs qui en font la demande.



2. **Manager/Worker Hiérarchique** : Un gestionnaire principal attribue des sous-ensembles de tâches à des sous-gestionnaires, qui les répartissent ensuite aux processeurs sous leur responsabilité.
3. **Schéma Décentralisé** : Chaque processeur possède son propre pool de tâches et peut demander du travail à d'autres processeurs lorsqu'il devient inactif.

L'objectif est d'évaluer et de comparer ces stratégies en termes d'efficacité et de scalabilité.

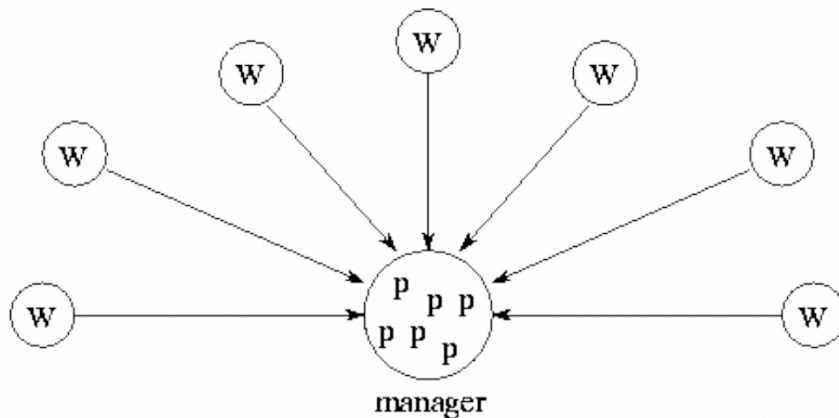


Figure 2.19: Manager/worker load-balancing structure. Workers repeatedly request and process problem descriptions; the manager maintains a pool of problem descriptions (p) and responds to requests from workers.

Questions

Manager/Worker

1. Écrivez un pseudo-code décrivant le fonctionnement du modèle Manager/Worker.
2. Le gestionnaire central maintient une file de tâches et répond aux requêtes des travailleurs en leur attribuant une tâche libre.
3. Analysez les avantages et inconvénients de cette approche en termes de charge de communication et de scalabilité.

Manager/Worker Hiérarchique

4. Modifiez l'algorithme précédent pour introduire des sous-gestionnaires qui attribuent les tâches à des groupes de processeurs.



-
5. Comparez cette approche avec le modèle simple Manager/Worker.

Schéma Décentralisé

6. Écrivez un pseudo-code d'un système où chaque processeur gère son propre pool de tâches et peut en demander à ses voisins en cas de besoin.
7. Discutez des avantages et inconvénients de ce modèle par rapport aux modèles centralisés.

Expérimentation et Comparaison

8. **Analysez la charge de communication** pour chaque approche et proposez des optimisations possibles.
Évaluez la robustesse de chaque modèle dans un scénario où certaines tâches prennent plus de temps que d'autres.
9. **Discutez du problème de détection de terminaison**, notamment dans les schémas décentralisés.

Partie Bonus : Implémentation en Python

- Simulez ces trois approches avec un nombre donné de processeurs et de tâches.
- Mesurez le temps d'exécution et le nombre de communications entre processeurs pour comparer leur efficacité.

Exercice 17. Modélisation de l'Atmosphère

Dans ce problème de modélisation de l'atmosphère, un modèle de grille est utilisé pour représenter l'état de l'atmosphère, et nous devons répartir les calculs associés à chaque point de la grille entre plusieurs processeurs. Le but est de développer des algorithmes parallèles qui maximisent la concurrence tout en réduisant les coûts de communication, en tenant compte de l'évolution dynamique des charges de travail.



Conservation of momentum:

$$\frac{du}{dt} - \left(f + u \frac{\tan \phi}{a} \right) v = - \frac{1}{a \cos \phi} \frac{1}{\rho} \frac{\partial p}{\partial \lambda} + F_\lambda$$

$$\frac{dv}{dt} + \left(f + u \frac{\tan \phi}{a} \right) u = - \frac{1}{\rho a} \frac{\partial p}{\partial \phi} + F_\phi$$

Hydrostatic approximation:

$$g = - \frac{1}{\rho} \frac{\partial p}{\partial z}$$

Conservation of mass:

$$\frac{\partial \rho}{\partial t} = - \frac{1}{a \cos \phi} \left(\frac{\partial}{\partial \lambda} (\rho u) + \frac{\partial}{\partial \phi} (\rho v \cos \phi) \right) - \frac{\partial}{\partial z} (\rho w)$$

Conservation of energy:

$$C_p \frac{dT}{dt} - \frac{1}{\rho} \frac{dp}{dt} = Q$$

State equation (atmosphere):

$$p = \rho R T$$

Figure 2.20: The basic predictive equations used in atmospheric modeling, where ϕ and λ are latitude and longitude, z is height, u and v are horizontal components of velocity, p is pressure, ρ is density, T is temperature, f is Coriolis force, g is gravity, F and Q are external forcing terms, C_p is specific heat, and a is the earth's radius.

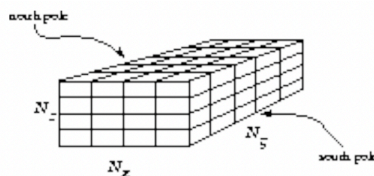


Figure 2.21: The three-dimensional grid used to represent the state of the atmosphere. Values maintained at each grid point represent quantities such as pressure and temperature.



*Un modèle atmosphérique est un programme informatique qui simule les processus atmosphériques (vent, nuages, précipitations, etc.) qui influencent le temps ou le climat. Il peut être utilisé pour étudier l'évolution des tornades, prédire le temps de demain, ou étudier l'impact de l'augmentation des concentrations de dioxyde de carbone dans l'atmosphère sur le climat. Comme de nombreux modèles numériques de processus physiques, un modèle atmosphérique résout un ensemble d'équations différentielles partielles, dans ce cas, décrivant le comportement de base de la dynamique des fluides de l'atmosphère (Figure 2.20). Le comportement de ces équations dans un espace continu est approximé par leur comportement sur un ensemble fini de points régulièrement espacés dans cet espace. En général, ces points sont situés sur une grille rectangulaire latitude-longitude de taille $N_x * N_y$, avec N_x dans la plage de 15 à 30, N_y dans la plage de 50 à 500 (Figure 2.21).*

Cette grille est périodique dans les dimensions x et y , ce qui signifie que le point de grille (x,y) est considéré comme étant adjacent à $(x+1,y)$ et $(x,y+1)$. Un vecteur de valeurs est maintenu à chaque point de grille, représentant des quantités telles que la pression, la température, la vitesse du vent et l'humidité.

Objectifs :

1. Partitionnement du Domaine :

- Le modèle de grille peut être partitionné selon les dimensions x , y et z . Vous devez déterminer la meilleure manière de diviser la grille pour maximiser la parallélisation. Proposer une partition du domaine pour un modèle de grille à trois dimensions.
 - Identifier les types de communications nécessaires entre les processeurs (stencils de différences finies, opérations globales, et calculs physiques).

2. Communication (Figure 2.23):

- La communication entre les processeurs est essentielle pour le calcul des stencils de différences finies, des opérations globales et des calculs physiques.
 - Décrire les types de communication nécessaires et comment elles peuvent être parallélisées.



- Discuter de la réduction des coûts de communication via l'**agglomération** des tâches, en particulier pour réduire les communications associées au stencil de neuf points et aux calculs physiques.
- **Stencils de différences finies.** Si l'on suppose une décomposition à grain fin dans laquelle chaque tâche encapsule un seul point de grille, le stencil à neuf points utilisé dans la dimension horizontale nécessite que chaque tâche obtienne des valeurs de huit tâches voisines. La structure de canal correspondante est illustrée dans la Figure 2.23. De même, le stencil à trois points utilisé dans la dimension verticale nécessite que chaque tâche obtienne des valeurs de deux voisins.
- **Opérations globales.** Le modèle atmosphérique calcule périodiquement la masse totale de l'atmosphère afin de vérifier que la simulation se déroule correctement. Cette quantité est définie comme suit :

$$Total\ Mass = \sum_{i=0}^{N_x-1} \sum_{j=0}^{N_y-1} \sum_{k=0}^{N_z-1} M_{ijk},$$

où M_{ijk} désigne la masse au point de grille (i,j,k). Cette somme peut être calculée en utilisant l'un des algorithmes de sommation parallèle présentés dans la Section 2.4.1.

- **Calculs de physique.** Si chaque tâche encapsule un seul point de grille, la composante physique du modèle atmosphérique nécessite une communication considérable. Par exemple, le ciel clair total (TCS) au niveau iii est défini comme suit :

$$\begin{aligned} TCS_k &= \prod_{i=1}^k (1 - cld_i) TCS_1 \\ &= TCS_{k-1} (1 - cld_k), \end{aligned}$$

où le niveau 0 correspond au sommet de l'atmosphère et cld_i est la fraction de nuages au niveau i .

3. Agglomération (Figure 24):



- L'agglomération des tâches consiste à regrouper plusieurs points de la grille par processeur afin de réduire le nombre de communications.
- Proposer une méthode d'agglomération (par exemple, de un à quatre points par tâche).
Analyser les avantages de l'agglomération pour la communication et l'ingénierie logicielle, notamment pour le composant physique du modèle.

4. Mapping et Charge Dynamique :

Une fois la partition et l'agglomération effectuées, il est nécessaire de définir la stratégie de mappage des tâches sur les processeurs.

Proposer une stratégie de mappage de type **SPMD** (Single Program Multiple Data), en tenant compte de la charge dynamique.

- Discuter des inconvénients des déséquilibres de charge dans la simulation et proposer une méthode de **mappage cyclique** pour réduire ces déséquilibres.
- Expliquer comment la variation des charges de travail (comme le calcul de radiation pendant le jour et les calculs de nuages) peut affecter la performance.

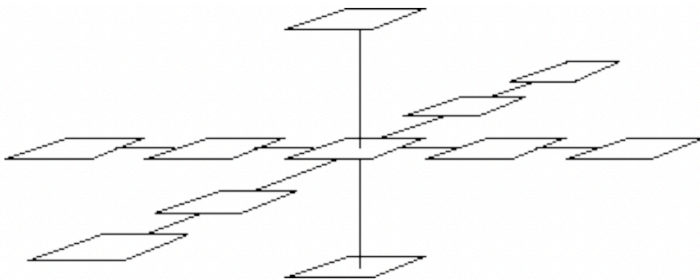


Figure 2.22: The finite difference stencils used in the atmosphere model. This figure shows for a single grid point both the nine-point stencil used to simulate horizontal motion and the three-point stencil used to simulate vertical motion.

Le modèle atmosphérique effectue une intégration temporelle pour déterminer l'état de l'atmosphère à un moment futur, basé sur un état initial. Cette intégration se fait par une série d'étapes, chaque étape faisant avancer l'état du calcul d'un montant fixe. On supposera que le modèle utilise une méthode de différences finies (Section 2.3.1) pour mettre à jour les valeurs de la grille, avec un stencil à neuf points utilisé pour simuler le



mouvement atmosphérique dans la dimension horizontale, et un stencil à trois points dans la dimension verticale (Figure 2.22).

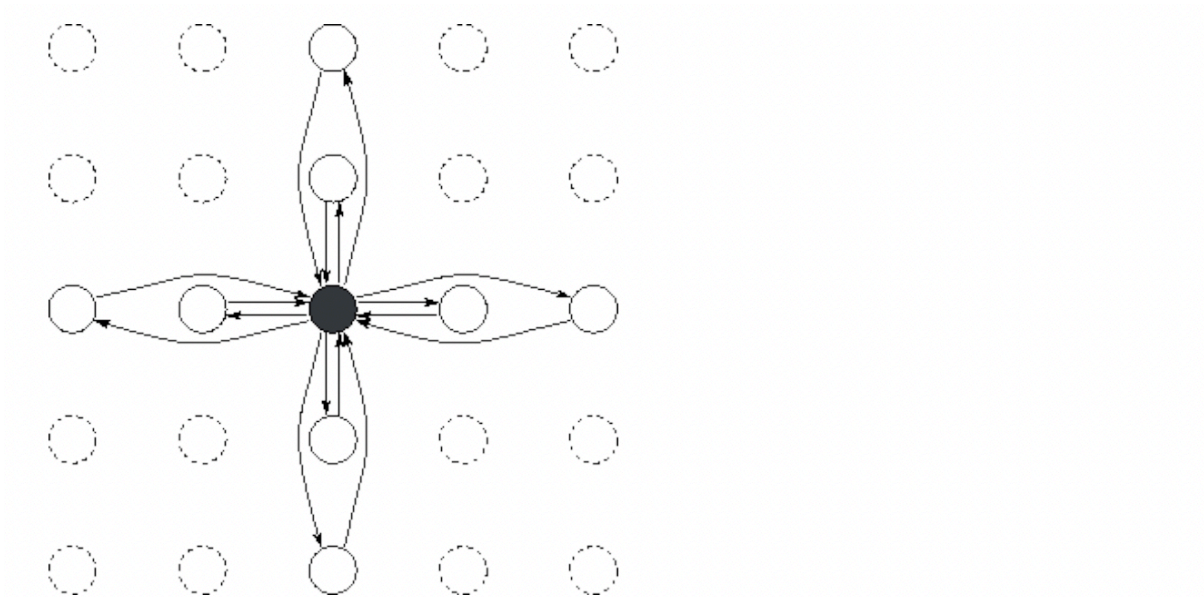


Figure 2.23: Task and channel structure for a two-dimensional finite difference computation with nine-point stencil, assuming one grid point per processor. Only the channels used by the shaded task are shown.

Questions :

1. Partition et Décomposition du Domaine

- Proposez un algorithme de partitionnement pour un modèle de grille $64 \times 128 \times 3$ dans lequel chaque tâche traite un point de la grille.
- Décrivez comment chaque processeur va gérer une sous-grille et quelles sont les communications nécessaires entre processeurs voisins pour effectuer les calculs de stencil.

2. Communication et Optimisation

- Décrivez comment les trois types de communication (stencils de différences finies, opérations globales, et calculs physiques) sont gérés en parallèle.
- Proposez des solutions pour réduire le nombre de communications et améliorer l'efficacité globale du calcul.



3. Agglomération des Tâches

- Expliquez comment augmenter la granularité des tâches (par exemple, de un à quatre points par tâche) et quel impact cela a sur la communication et l'efficacité du calcul.
- Donnez un exemple de comment l'agglomération réduit le nombre de messages dans le stencil de neuf points.

4. Stratégie de Mappage et Gestion de la Charge Dynamique

- Proposez un mappage de type SPMD en utilisant une **partition horizontale**.
Discutez de l'impact des déséquilibres de charge en fonction des conditions diurnes/nocturnes et de la variabilité de la charge de travail des processeurs.

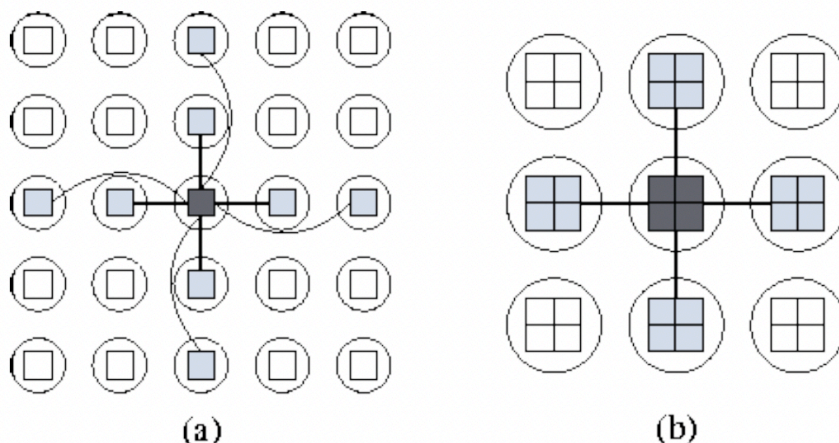


Figure 2.24: Using agglomeration to reduce communication requirements in the atmosphere model. In (a), each task handles a single point and hence must obtain data from eight other tasks in order to implement the nine-point stencil. In (b), granularity is increased to 2×2 points, meaning that only 4 communications are required per task.

Bonus : Simulation et Analyse des Performances

- **Implémentation en Python (ou autre langage de votre choix) :**
 - Simulez un petit modèle de grille (par exemple, 16×16) en utilisant différentes stratégies de partition et de mappage.
Mesurez l'impact de l'agglomération et du mappage cyclique sur le temps d'exécution et la répartition de la charge de travail.



-
- Comparez les résultats avec et sans l'utilisation du mappage cyclique pour réduire les déséquilibres de charge.